Observability — Interview Ready Answers

Comprehensive answers with examples for all 5 Observability questions

5 Questions • Incident Triage • Alerting & PagerDuty • Distributed Tracing • SRE & Error Budgets • Post-Mortems

Q1. How do you trace a spike in failed transactions to either app logic, infrastructure failure, or a third-party API?

Answer

Transaction failure spikes require a disciplined, layered investigation that uses all three pillars of observability — metrics, logs, and traces — to pinpoint whether the fault originates in your application code, your infrastructure, or an upstream external dependency. Jumping to conclusions without evidence wastes time and leads to the wrong fix.

The Three-Pillar Investigation Framework

```
Symptoms:  Error rate spike → Transaction failures

Pillar 1 — Metrics:  WHEN did it start? What resources are affected?
Pillar 2 — Logs:     WHAT error? Which code path? Stack trace?
Pillar 3 — Traces:   WHERE in the call chain? App, DB, or external API?
```

Step 1 — Establish the Timeline (Metrics)

Before investigating root cause, establish the exact moment the spike began and correlate it with any changes.

```
# Prometheus — error rate query
rate(http_requests_total{status=~'5..'}[1m])
  / rate(http_requests_total[1m])

# Grafana — overlay multiple signals on one graph:
# - Error rate (line 1)
# - Deployment events (annotations)
# - Infrastructure events (node replacement, scaling)
# - Third-party API latency (if instrumented)
```

If the spike aligns exactly with a deployment: app logic or a misconfiguration is the likely cause. If the spike begins on its own with no deployment: infrastructure or third-party failure.

Step 2 — Characterise the Failure (Metrics + Logs)

Hypothesis A — Application Logic Failure

Signs: 4xx or 5xx errors with a specific stack trace pattern. Error appears consistently on a specific endpoint. CPU/memory normal. Infrastructure healthy.

```
# CloudWatch Logs Insights — find error pattern
fields @timestamp, level, message, errorType
| filter level = 'ERROR'
| filter @timestamp > ago(30m)
| stats count(*) as errorCount by errorType
| sort errorCount desc

# Example output:
# NullPointerException     342
# ValidationException       28
# → App logic issue — specific exception in your code

# Structured log query (Kibana / Loki)
{service='payment-api'} |= 'ERROR' | json
  | line_format '{{.error}} {{.stacktrace}}'
```

Hypothesis B — Infrastructure Failure

Signs: Generic connection errors, timeouts. Multiple endpoints affected simultaneously. Node metrics show issues. Pod restarts visible in Kubernetes.

```
# Check node health
kubectl get nodes
kubectl describe node <node>  # Look for MemoryPressure, DiskPressure

# Check pod restarts
kubectl get pods --all-namespaces --sort-by='.status.containerStatuses[0].restartCount'

# Check OOM kills
```

```
dmesg | grep -i 'oom\|killed'
journalctl -k | grep 'Out of memory'

# Database connection pool exhaustion (common infra cause)
SELECT count(*) FROM pg_stat_activity WHERE state = 'active';
# If near max_connections: infra bottleneck

# Prometheus — infra metrics
# CPU: avg(rate(node_cpu_seconds_total{mode!='idle'}[5m])) by (node) > 0.85
# Memory: node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes < 0.1
# Disk I/O: rate(node_disk_io_time_seconds_total[5m]) > 0.8
```

Hypothesis C — Third-Party API Failure

Signs: Errors concentrated in specific service calls. Latency spikes on outbound requests. Error messages reference external service. Your other endpoints work fine.

```
# Check your HTTP client metrics for outbound calls
# If using Prometheus with HTTP client instrumentation:
histogram_quantile(0.99,
  rate(http_client_request_duration_seconds_bucket{
    target_host='api.stripe.com'}[5m])
)
# Spike here → third-party is slow

# Check third-party error responses in your logs
grep 'api.stripe.com\|stripe\|external' /var/log/app/app.log \
  | grep -i 'timeout\|error\|503\|429'

# Check status pages
# status.stripe.com, status.twilio.com, etc.
# Correlate their incident time with your spike
```

Step 3 — Distributed Traces for Precise Location

Distributed traces show the exact call chain — where time is spent and where errors originate. A single failed transaction trace tells you more than a thousand metrics:

```
# Jaeger / Tempo — find failed traces
# Query: service=payment-api, tags: error=true, duration > 2s

# Example trace analysis:
payment-api (12ms) → auth-service (8ms) → stripe-api (28,435ms) ← TIMEOUT
# Root cause: Stripe API timing out after 28 seconds
# Your service is a victim, not the cause

# Another trace:
payment-api (5ms) → database (0ms) ← CONNECTION REFUSED
# Root cause: Database connection refused — infra issue

# Another trace:
payment-api: NullPointerException at PaymentValidator.java:127
# Root cause: App logic — null check missing
```

Decision Tree — Root Cause Identification

Signal Pattern	Most Likely Root Cause
Stack trace in logs, specific endpoint	App logic / code bug
Multiple endpoints fail, node metrics degraded	Infrastructure failure
Latency spike on outbound calls only	Third-party API degraded
Errors started at exact deployment time	New code regression
Database error (connection refused, timeout)	Database or connection pool issue
Errors on specific pod subset, others fine	Single node/pod issue

Real-World Example

'Payment transaction failures spiked from 0.2% to 14% at 14:32. Grafana showed the spike started precisely at 14:30 — 2 minutes after a deployment. Traces showed all failures had the same root span: a NullPointerException in PaymentValidator when processing credit card CVV for non-UK cards. The UK-only feature flag had been accidentally left off in the new release, passing null CVV to the validator. Helm rollback at 14:41 restored error rate to baseline. Total impact: 9 minutes.'

> *Tip: Always check deployment events first — the majority of production spikes occur within minutes of a deployment. Build a 'deployment dashboard' that overlays error rate, latency p99, and deployment annotations on one graph. This single view answers the most common question: 'Did something change just before the spike?'*

Q2. What's your approach to building actionable alerts and reducing false positives in PagerDuty/Prometheus?

Answer

Alert fatigue is one of the most serious operational problems in engineering. When engineers receive too many alerts — especially ones that don't require action — they start ignoring them. A single false positive that wakes someone at 3 AM and turns out to be nothing can erode trust in the entire alerting system. Building actionable alerts means: every alert that fires requires human attention and a specific response.

The Golden Rule — Every Alert Must Have an Immediate Human Action

Before creating any alert, answer these questions:

- Is a human required to respond right now, or can this wait until morning?
- What specific action should the engineer take when this fires?
- Will this alert fire in a way that disrupts someone's sleep for no real impact?

If the answer to the third question is 'maybe,' the alert needs refinement before deployment.

Alert Design Principles

1. Alert on Symptoms, Not Causes

Users experience symptoms (slow page loads, transaction failures) — not causes (high CPU, low memory). Alert on what users experience:

```
# WRONG — alerts on cause (CPU). CPU being high doesn't always mean users are impacted
alert: HighCPU
  expr: node_cpu_utilization > 0.85
  for: 5m

# RIGHT — alert on symptom (error rate). Users are definitely experiencing this
alert: HighTransactionErrorRate
  expr: |
    rate(http_requests_total{status=~'5..', service='payment-api'}[5m])
    / rate(http_requests_total{service='payment-api'}[5m]) > 0.01
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: 'Payment API error rate above 1% for 2 minutes'
    description: 'Current rate: {{ $value | humanizePercentage }}. Run: kubectl logs -l
app=payment-api -n production | grep ERROR'
    runbook_url: 'https://wiki.company.com/runbooks/payment-api-errors'
```

2. Use the SLO/Error Budget Approach

Alert on error budget burn rate rather than absolute thresholds. Error budget burning too fast means users are being impacted faster than acceptable:

```
# SLO: 99.9% availability = 43.8 minutes/month error budget

# Multiwindow, multi-burn-rate alerting (Google SRE Book approach)
# Alert when burning error budget faster than acceptable rate

# Page immediately: burning budget 14x faster (1h window + 5m window)
# → Would exhaust monthly budget in ~2 hours
- alert: ErrorBudgetBurnRateFast
  expr: |
    (
      rate(http_requests_total{status=~'5..'}[1h])
      / rate(http_requests_total[1h])
```

```
    ) > 14 * 0.001    # SLO violation rate × burn-rate multiplier
    AND
    (
      rate(http_requests_total{status=~'5..'}[5m])
      / rate(http_requests_total[5m])
    ) > 14 * 0.001
  for: 2m
  labels:
    severity: critical
    slo: payment-availability

# Ticket only: burning 3x faster (6h window + 30m window)
# → Would exhaust monthly budget in ~10 days
- alert: ErrorBudgetBurnRateSlow
  expr: |
    rate(http_requests_total{status=~'5..'}[6h])
    / rate(http_requests_total[6h]) > 3 * 0.001
  labels:
    severity: warning
```

3. Tune the 'for' Duration to Eliminate Transient Spikes

A single anomalous data point should not wake an engineer. The 'for' duration ensures the condition persists before firing:

```
# Bad: fires on a single data point (often a transient blip)
alert: HighLatency
  expr: http_request_duration_p99 > 2
  # No 'for' duration — fires immediately on one sample

# Good: must persist for 5 minutes before firing
alert: HighLatency
  expr: |
    histogram_quantile(0.99,
      rate(http_request_duration_seconds_bucket[5m])) > 2
  for: 5m   # Must be consistently elevated, not a one-off spike
```

4. Include Actionable Runbook Links in Every Alert

```
annotations:
  summary: 'Payment service p99 latency above 2 seconds'
  description: |
    Service: {{ $labels.service }}
    Current p99: {{ $value }}s
    Impact: Users experiencing slow checkout
  runbook_url: 'https://wiki.company.com/runbooks/payment-latency'
  dashboard_url: 'https://grafana.company.com/d/payment-overview'
```

5. Severity Tiering — Match Alert to Response

Severity	Condition
Critical / Page	User impact NOW — SLO burning fast
Warning / Ticket	Degraded but within budget — monitor
Info	Normal variance — FYI only
None	Informational metric threshold

6. PagerDuty Configuration — Routing and Escalation

```
# PagerDuty service configuration
Service: Payment API

Routing rules:
  - condition: severity == 'critical' AND service == 'payment-api'
    action: Page on-call engineer immediately

  - condition: severity == 'warning'
    action: Create low-urgency alert (business hours only)

Escalation policy:
  Level 1: On-call engineer (immediate)
  Level 2: Team lead (if no acknowledgement in 10 minutes)
```

```
  Level 3: Engineering manager (if no acknowledgement in 20 minutes)

Schedules:
  Weekday on-call: rotating weekly among 4 engineers
  Weekend on-call: separate weekend rotation
  Override: manual escalation for critical incidents
```

7. Reducing False Positives — Practical Techniques

- Use rate() and histogram_quantile() — Don't alert on raw counters. rate() smooths the query over time and reduces spike sensitivity.
- Group and deduplicate with Alertmanager — If 20 pods have the same issue, send one alert, not 20:

```
# Alertmanager route — group related alerts
route:
  group_by: [alertname, service, namespace]
  group_wait: 30s      # Wait to batch related alerts
  group_interval: 5m   # Re-notify interval
  repeat_interval: 4h  # Don't re-page if already acknowledged
```

- Inhibition rules — When a node is down, don't send alerts for every pod on that node:

```
# If node is down, suppress pod-level alerts
inhibit_rules:
  - source_match:
      alertname: NodeDown
    target_match:
      alertname: PodNotReady
    equal: [node]    # Same node
```

- Review alert history — Weekly, look at which alerts fired most. The top 5 most-fired alerts with low acknowledgement rate are your false positive candidates.
- Alert-on-absence patterns — Use absent() for alerts that should always be present:

```
# Alert if no metrics from payment service for 5 minutes
- alert: PaymentServiceMetricsMissing
  expr: absent(up{job='payment-api'} == 1)
  for: 5m
```

Alert Quality Metrics — Track These

Metric	Target
Alert signal-to-noise ratio	< 10% of pages are false positives
Mean time to acknowledge (MTTA)	< 5 minutes
Alert volume per engineer per week	< 5 critical pages
Percentage of actionable alerts	> 90%
Alert-to-incident correlation	> 85%

> *Tip: Conduct a quarterly 'alert review' session. Pull the last 90 days of alert history from PagerDuty and Alertmanager. For every alert that fired more than 5 times, ask: was action always required? Was the action always the same? Could this be automated? This systematic review eliminates false positives and improves alert quality faster than any tooling change.*

Q3. How would you implement distributed tracing in a Kubernetes-based microservices system?

Answer

Distributed tracing connects the dots across multiple services in a microservices architecture — allowing you to follow a single request as it flows through 10+ services, identify which service introduced latency, and pinpoint the exact line of code where errors occur. In Kubernetes, implementing tracing requires: instrumentation at the application level, a collector/agent to receive spans, a backend storage system, and a UI for querying and visualising traces.

The Three Pillars of Distributed Tracing

Concept	Definition
Trace	The entire end-to-end

	journey of one request
Span	One unit of work within a trace — one service's contribution
Context propagation	Passing trace ID through all service calls

Architecture — OpenTelemetry + Tempo + Grafana

```
Application Services
  | (OpenTelemetry SDK — auto or manual instrumentation)
  ↓
OpenTelemetry Collector (DaemonSet or Deployment in K8s)
  | (receives spans via OTLP — gRPC or HTTP)
  | (processes: batch, filter, sample)
  ↓
Grafana Tempo (trace backend — scalable, cost-effective)
  | (stores traces)
  ↓
Grafana (trace UI — query, visualise, link to logs and metrics)
```

Step 1 — Deploy OpenTelemetry Operator

```
# Install cert-manager (required by OTel operator)
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true

# Install OpenTelemetry Operator
helm repo add open-telemetry https://open-telemetry.github.io/opentelemetry-helm-charts
helm install opentelemetry-operator open-telemetry/opentelemetry-operator \
  --namespace opentelemetry-system --create-namespace
```

Step 2 — Deploy OpenTelemetry Collector

```
# OpenTelemetryCollector resource — DaemonSet mode
apiVersion: opentelemetry.io/v1alpha1
kind: OpenTelemetryCollector
metadata:
  name: otel-collector
  namespace: monitoring
spec:
  mode: daemonset    # One collector pod per node
  config: |
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318

    processors:
      batch:
        timeout: 1s
        send_batch_size: 1024
      resource:
        attributes:
        - key: k8s.cluster.name
          value: prod-eks
          action: upsert
      # Tail-based sampling — keep 100% of error traces, 10% of success
      tail_sampling:
        decision_wait: 10s
        policies:
        - name: errors-policy
          type: status_code
          status_code: { status_codes: [ERROR] }
        - name: slow-traces
          type: latency
          latency: { threshold_ms: 1000 }
        - name: probabilistic-sampling
```

```
        type: probabilistic
        probabilistic: { sampling_percentage: 10 }

  exporters:
    otlp:
      endpoint: 'http://tempo.monitoring:4317'
      tls:
        insecure: true

  service:
    pipelines:
      traces:
        receivers: [otlp]
        processors: [resource, batch, tail_sampling]
        exporters: [otlp]
```

Step 3 — Deploy Grafana Tempo

```
# values.yaml for Tempo Helm chart
helm repo add grafana https://grafana.github.io/helm-charts
helm install tempo grafana/tempo-distributed \
  --namespace monitoring \
  --set storage.trace.backend=s3 \
  --set storage.trace.s3.bucket=my-tempo-traces \
  --set storage.trace.s3.region=us-east-1
```

Step 4 — Auto-Instrument Applications (Zero Code Change)

The OpenTelemetry Operator supports auto-instrumentation — it injects an OpenTelemetry SDK sidecar or init container into pods automatically based on annotations. No application code change required:

```
# Enable auto-instrumentation for a namespace
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: auto-instrumentation
  namespace: production
spec:
  exporter:
    endpoint: http://otel-collector.monitoring:4317
  propagators:
    - tracecontext
    - baggage
  sampler:
    type: parentbased_traceidratio
    argument: '0.1'    # Sample 10% of traces
  nodejs:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-nodejs:latest
  python:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-python:latest
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-java:latest

# Annotate pods to enable auto-instrumentation
# In your deployment template:
spec:
  template:
    metadata:
      annotations:
        instrumentation.opentelemetry.io/inject-nodejs: 'true'
        # or: inject-python: 'true'  inject-java: 'true'
```

Step 5 — Manual Instrumentation for Custom Spans

For business-critical paths (payment processing, fraud checks), add custom spans to capture application-level context:

```
// Node.js manual instrumentation
const { trace, SpanStatusCode } = require('@opentelemetry/api');

async function processPayment(paymentData) {
  const tracer = trace.getTracer('payment-service');

  return await tracer.startActiveSpan('process-payment', async (span) => {
    try {
      // Add business context as span attributes
      span.setAttributes({
```

```
        'payment.currency':    paymentData.currency,
        'payment.amount':      paymentData.amount,
        'payment.method':      paymentData.method,
        'customer.country':    paymentData.customerCountry,
      });

      // Child span for external Stripe call
      return await tracer.startActiveSpan('stripe-charge', async (stripeSpan) => {
        stripeSpan.setAttributes({'stripe.idempotency_key': paymentData.idempotencyKey});
        const result = await stripe.charges.create({...paymentData});
        stripeSpan.setAttributes({'stripe.charge_id': result.id});
        stripeSpan.end();
        return result;
      });
    } catch (error) {
      span.setStatus({ code: SpanStatusCode.ERROR, message: error.message });
      span.recordException(error);
      throw error;
    } finally {
      span.end();
    }
  });
}
```

Step 6 — Context Propagation Between Services

Trace context must be passed in HTTP headers between services. The W3C TraceContext standard uses the traceparent header. OpenTelemetry auto-instrumentation handles this automatically for HTTP/gRPC calls:

```
# HTTP request with trace context header
GET /api/v1/orders HTTP/1.1
Host: order-service
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
# Format: version-traceId-parentSpanId-traceFlags

# For manual HTTP calls — propagate context:
const { propagation, context } = require('@opentelemetry/api');
const headers = {};
propagation.inject(context.active(), headers);
await fetch('http://order-service/api/v1/orders', { headers });
```

Step 7 — Connect Traces to Logs (Exemplars)

Link traces to logs using the trace ID. This allows jumping from a trace directly to the log lines generated by that specific request:

```
// Structured logging — include trace ID
const { trace } = require('@opentelemetry/api');

function log(level, message, extra = {}) {
  const activeSpan = trace.getActiveSpan();
  const traceId = activeSpan ? activeSpan.spanContext().traceId : '';
  const spanId  = activeSpan ? activeSpan.spanContext().spanId  : '';

  console.log(JSON.stringify({
    timestamp: new Date().toISOString(),
    level, message,
    traceId, spanId,   // ← Link to trace
    ...extra
  }));
}

# Grafana — click 'Logs for this span' in trace view
# → Jumps to Loki/CloudWatch logs filtered by traceId
```

Step 8 — Configure Grafana Data Sources

```
# In Grafana — link Tempo and Loki for trace-to-log correlation
# Tempo data source settings:
#   URL: http://tempo.monitoring:3100
#   Derived fields:
#     Name: TraceID
#     Regex: traceId=(\w+)
#     Internal link: Tempo

# Loki data source settings:
```

```
#   Derived fields:
#     Name: TraceID
#     Regex: traceId=(\w+)
#     Internal link: Tempo
#     URL: http://tempo.monitoring:3100/trace/$${__value.raw}
```

Sampling Strategy — Don't Trace Everything

Strategy	Description
Head-based (probability)	Sample N% of requests at trace start
Tail-based	Decide to keep after trace completes — keep errors, slow traces
Rate-limited	Fixed N traces per second regardless of traffic
Always sample errors	Keep 100% of error traces, sample success

Practical Trace Queries — Grafana Tempo TraceQL

```
# Find all traces > 2 seconds from payment-api in the last hour
{ .service.name = "payment-api" && duration > 2s }

# Find error traces from a specific user
{ .service.name = "payment-api" && status = error && .customer.id = "cust_123" }

# Find traces involving stripe-api with latency breakdown
{ .service.name = "payment-api" } | select(span:childOf(.), .service.name)
```

> *Tip: Start with auto-instrumentation (zero code changes) to get traces from all services immediately. Then add manual instrumentation for business-critical spans (payment processing, fraud checks) to capture domain context that auto-instrumentation can't provide. This phased approach delivers value in days, not weeks.*

Q4. Explain SRE principles and how you would implement error budgets in your organization.

Answer

Site Reliability Engineering (SRE) is a discipline developed at Google that applies software engineering principles to infrastructure and operations problems. Its fundamental insight is that reliability is a feature — one that must be balanced against the speed of feature delivery. Error budgets are the mechanism that makes this balance concrete, measurable, and actionable.

Core SRE Principles

1. Embracing Risk — The Reliability Spectrum

Perfect reliability (100% uptime) is impossible and, more importantly, undesirable. Every minute spent achieving the last 0.001% of uptime is a minute not spent on features users actually want. The SRE approach accepts this explicitly:

SLO Target	Monthly Downtime Allowed
99%	7h 18m
99.9%	43m 49s
99.95%	21m 54s
99.99%	4m 22s
99.999%	26 seconds

2. Service Level Objectives (SLOs) — The Foundation

An SLO is a target reliability level for a specific metric, measured over a rolling time window. SLOs are the contract between the engineering team and the business:

```
# Well-defined SLO format
Service: payment-api
SLI (what we measure): successful_requests / total_requests
SLO (the target):      99.9% over a 30-day rolling window
```

```
Consequence if missed: error budget exhausted → deployment freeze

# Multiple SLOs per service
Availability SLO:  99.9% of requests return 2xx in 30 days
Latency SLO:       95% of requests complete in < 300ms
                99% of requests complete in < 1000ms
```

Service Level Indicators (SLIs) — What to Measure

Service Type	Recommended SLIs
API / Web service	Request success rate, request latency (p50, p95, p99)
Data pipeline	Freshness (data age), correctness (data quality), throughput
Storage / Database	Durability (data intact), availability, read/write latency
Batch job	Completion rate, time to complete, data accuracy

3. Error Budgets — Making the Reliability Trade-Off Concrete

An error budget is the amount of unreliability you're allowed to have in a time period, as defined by your SLO. If your SLO is 99.9%, your monthly error budget is 0.1% of all requests — approximately 43 minutes of total downtime.

```
Error Budget Calculation:
  SLO:           99.9%
  Time window:   30 days = 43,200 minutes
  Error budget:  0.1% = 43.2 minutes

  If you have 200,000 requests/day = 6,000,000 requests/month
  Error budget:  0.1% × 6,000,000 = 6,000 failed requests allowed

  After a 20-minute outage that caused 2,000 failures:
  Budget remaining: 4,000 failures / 6,000 = 66.7% of budget left
  Budget burned:    33.3%
```

4. Error Budget Policy — The Consequences

The error budget only works if there are defined consequences for burning it. The policy must be agreed before an incident, not invented after:

```
Error Budget Policy (example):

  Budget remaining > 50%:
    → Full deployment velocity. Experiment freely.
    → Reliability work is optional but encouraged.

  Budget remaining 25-50%:
    → Warning. Review upcoming deployments.
    → Prioritise reliability fixes in next sprint.
    → Requires tech lead sign-off on risky changes.

  Budget remaining 0-25%:
    → Feature deployment freeze. No new features to production.
    → Only reliability fixes, rollbacks, and hotfixes allowed.
    → Entire team focuses on reliability work until budget recovers.

  Budget exhausted (< 0%):
    → Emergency reliability work mode.
    → Incident post-mortem required within 24 hours.
    → Full deployment freeze until SLO is met for 72 consecutive hours.
```

Implementing Error Budgets — Step by Step

Step 1 — Define SLOs with Business Stakeholders

SLOs should be set based on what users actually need, not what the engineering team thinks is achievable. Run a joint session with product, business, and engineering:

- What does a bad user experience look like? When do users churn or complain?
- How much reliability are we willing to invest in? (More reliability = fewer features)
- What is our competitive baseline? (What do competitors offer?)

Step 2 — Build Error Budget Dashboard in Grafana

```
# Prometheus recording rules for error budget
groups:
  - name: error_budget
```

```
  rules:
  # Error rate over 30 days
  - record: job:http_request_error_rate:30d
    expr: |
      (
        sum(increase(http_requests_total{status=~'5..'}[30d]))
        /
        sum(increase(http_requests_total[30d]))
      )

  # Error budget remaining percentage
  - record: job:error_budget_remaining:30d
    expr: |
      1 - (job:http_request_error_rate:30d / (1 - 0.999))
      # (1 - 0.999) = SLO error rate allowance (0.001 = 0.1%)

  # Error budget burn rate (14x = critical, will exhaust in ~2 hours)
  - record: job:error_budget_burn_rate:1h
    expr: |
      (
        rate(http_requests_total{status=~'5..'}[1h])
        / rate(http_requests_total[1h])
      ) / (1 - 0.999)
```

Step 3 — Alerting on Error Budget Burn Rate

```
# Multi-window burn rate alerts (Google SRE recommendation)
groups:
- name: slo_alerts
  rules:
  # Critical: burning budget 14x faster — page immediately
  - alert: ErrorBudgetBurnRateCritical
    expr: |
      job:error_budget_burn_rate:1h > 14
      AND
      job:error_budget_burn_rate:5m > 14
    for: 2m
    labels:
      severity: critical
      page: 'true'
    annotations:
      summary: 'Error budget burning 14x faster than allowed'
      description: 'At this rate, monthly budget exhausted in ~2 hours'
      runbook_url: 'https://wiki.company.com/runbooks/slo-budget-burn'

  # Warning: burning 3x faster — create ticket, investigate tomorrow
  - alert: ErrorBudgetBurnRateHigh
    expr: |
      job:error_budget_burn_rate:6h > 3
      AND
      job:error_budget_burn_rate:30m > 3
    for: 15m
    labels:
      severity: warning
    annotations:
      summary: 'Error budget burning 3x faster than allowed'
      description: 'At this rate, monthly budget exhausted in ~10 days'
```

Step 4 — Sprint Reliability Reviews

Every sprint planning session should include an error budget review:

- How much budget was burned last sprint?
- Was it from incidents? Deployments? Expected degradations?
- How much budget do we have left for the next sprint?
- Based on the budget, what reliability work should we include in this sprint?

SRE Toil Reduction

Toil is manual, repetitive operational work that doesn't produce lasting value. SRE teams target keeping toil below 50% of their working time. Examples of toil and its automation:

Toil	Automated Replacement
Manual certificate renewal	cert-manager auto-rotation

Manual scaling during traffic spikes	HPA + Karpenter auto-scaling
Manually restarting crashed pods	Kubernetes liveness probes + restartPolicy
Manual log rotation	logrotate systemd + Fluent Bit lifecycle
Manually provisioning dev environments	Terraform + Atlantis on PR

> *Tip: Start with one SLO for your most important service and build the full error budget system around it — dashboard, alerts, and policy. Getting the organisation to agree on consequences is the hardest part. Once one team operates with error budgets successfully, adoption across other teams happens naturally because engineers see the benefit: data-driven deployment authority instead of politics-based release gates.*

Q5. Walk me through a post-mortem process. How do you conduct effective incident reviews?

Answer

A post-mortem (also called an incident review or after-action review) is a structured analysis conducted after a production incident. Its purpose is not to assign blame but to understand what happened, why it happened, and how to prevent recurrence. The quality of your post-mortem process is a direct indicator of your engineering organisation's maturity.

The Blameless Post-Mortem Principle

The most important cultural foundation for effective post-mortems is blamelessness. When engineers fear punishment for mistakes, they hide information, avoid taking responsibility, and incidents are investigated incompletely. The blameless approach assumes that:

- Engineers always act with good intentions given the information available to them at the time.
- Human error is a symptom, not a cause. The real cause is a system that allowed human error to have an outsized impact.
- The goal is to improve the system, not to punish individuals.

Post-Mortem Timeline — When to Conduct

```
Incident occurs → Incident managed → Service restored
                                          ↓
                          24-48 hours: Post-mortem meeting
                                          ↓
                          48-72 hours: Post-mortem document published
                                          ↓
                          Sprint+1: Action items in backlog
                                          ↓
                          30 days: Review action item completion
```

The Post-Mortem Document Structure

Section 1 — Incident Summary

```
Title:      Payment API Outage — 2024-01-19
Date:       2024-01-19 14:32 UTC
Duration:   22 minutes (14:32 – 14:54)
Severity:   P1
Impact:     95% of payment transactions failed
            Approximately 8,400 users affected
            Estimated revenue impact: ~£42,000
Status:     Resolved — action items in progress
Authors:    @alice (IC), @bob (tech lead), @carol (on-call)
```

Section 2 — Timeline (The Most Important Section)

The timeline must be reconstructed from facts — logs, metrics, alerts, Slack messages. Each entry has an exact timestamp and the source of the information:

```
14:30:00  Deployment of payment-api v2.1.0 begins (Helm upgrade)
14:32:15  First 5xx errors appear in logs (CloudWatch)
14:32:40  Error rate crosses 1% threshold — PagerDuty alert fires
14:33:12  Alice acknowledges PagerDuty alert
14:33:30  Alice opens #incident-2024-01-19 channel, pages Bob
14:35:00  Bob joins — identifies 5xx pattern in logs: NullPointerException
14:36:00  Hypothesis: new deployment caused the regression
14:38:00  Decision to rollback payment-api to v2.0.8
14:38:30  Helm rollback initiated
14:41:00  Rollback complete — pods healthy
14:41:30  Error rate drops from 95% to 0.3%
14:43:00  Error rate stable at baseline (0.2%)
```

```
14:54:00  Carol confirms no further anomalies — incident resolved
14:54:00  Status page updated: resolved
```

Section 3 — Root Cause Analysis

The root cause analysis uses the '5 Whys' technique — iteratively asking 'why' until you reach the systemic root cause, not just the immediate trigger:

```
Immediate cause:
  payment-api v2.1.0 caused NullPointerException on non-UK card payments

Why did NullPointerException occur?
  → The new CVV validation code assumed CVV was never null,
     but non-UK cards don't require CVV

Why was this not caught in testing?
  → Integration tests only used UK test card fixtures (with CVV)
     No test case covered non-UK cards without CVV

Why were non-UK cards excluded from test fixtures?
  → The original feature was UK-only. When expanded globally,
     test fixtures were not updated

Why was test fixture coverage not reviewed?
  → No checklist requirement to review test fixtures when expanding
     feature scope to new regions

Root cause:
  Absence of test coverage for non-UK card processing combined with
  missing defensive null checks in payment validation code.
  The deployment process had no gate that verified international
  card processing before production deployment.
```

Section 4 — What Went Well

Identifying what went well is not window dressing — it reinforces effective practices and prevents them from being accidentally removed in system changes:

- PagerDuty alert fired within 25 seconds of the first errors — fast detection.
- On-call engineer acknowledged in 32 seconds — within SLA.
- Incident channel created within 90 seconds — fast communication.
- Helm rollback was well-practiced — completed in 2.5 minutes.
- Status page was updated within 3 minutes of incident declaration.

Section 5 — What Went Poorly

Be specific and factual — not personal. The phrase 'The system had…' rather than 'Bob forgot to…' keeps the analysis blameless:

- The deployment pipeline did not have an international card smoke test — the gap allowed a regression to reach production.
- The 22-minute duration was longer than our 15-minute MTTR target — rollback took 8 minutes longer than estimated due to a slow Kubernetes drain on one node.
- Stakeholders were not notified proactively — the customer success team learned about the incident from user complaints, not from an internal alert.

Section 6 — Action Items (The Most Valuable Output)

Each action item must have: a specific owner, a due date, a JIRA ticket, and a type (prevent recurrence, reduce detection time, reduce recovery time):

Action	Owner
Add null-safety checks to all payment validation code	Alice
Add international card test fixtures to integration test suite	Bob
Add non-UK card smoke test to staging deployment	Alice

gate	
Fix slow node drain — identify cause of 5-minute delay	Carol
Set up stakeholder alert for P1 incidents (auto-notify CS team)	Carol
Add CVV nullability to PR review checklist	Bob

Section 7 — Metrics

```
Detection time:    25 seconds (alert fired)
Acknowledge time:  32 seconds (on-call responded)
Recovery time:     22 minutes (service restored)
Error budget spent: 33.3% of monthly budget
Revenue impact:    ~£42,000
Users affected:    ~8,400
```

Running the Post-Mortem Meeting Effectively

- Duration: 60 minutes maximum — longer meetings lose focus.
- Attendees: Incident commander, technical participants, one stakeholder representative. Not the entire company.
- Facilitation: Separate the facilitator from the incident commander. Facilitators keep the conversation blameless and on-track.
- Timeline first: Spend 20 minutes reconstructing the timeline together. Everyone fills in gaps. Establishes shared facts.
- 5 Whys together: Conduct the root cause analysis in the room — don't let one person write it alone.
- Action items: Every action item must leave the room with an owner and a due date. Unowned action items never get done.

Post-Mortem Quality Checklist

Criterion	Good
Root cause analysis	5-level causal chain to systemic root cause
Timeline	Exact timestamps from logs/metrics
Action items	Specific, owned, dated, ticketed
Blamelessness	System-level framing throughout
What went well	Specific practices identified
Distribution	Published to all engineers within 72 hours

Building a Post-Mortem Culture

- Publish all post-mortems — make them searchable by all engineers. Learning should be shared, not siloed.
- Celebrate post-mortems — teams that write detailed, honest post-mortems are doing the right thing. Recognise this publicly.
- Review quarterly — pick 2-3 historical post-mortems quarterly and ask: are the action items done? Did they prevent recurrence?
- Track action item completion — a post-mortem with no completed action items is just a report. Assign a DRI (Directly Responsible Individual) for each action item, not just a team.

> *Tip: The single most important indicator of post-mortem quality is the action item completion rate. Track it monthly. If < 70% of post-mortem action items are completed within the agreed timeline, the root problem is not the quality of the analysis — it is that reliability work is not prioritised in sprint planning. The error budget policy (Q4) directly solves this by creating a data-driven forcing function for reliability investment.*
